

Berikut adalah penjelasan mendalam mengenai kode C# tersebut. Kode ini adalah implementasi fitur **Konvolusi Citra (Image Convolution)** untuk aplikasi pengolah gambar (seperti Photoshop sederhana).

Kode ini dibagi menjadi dua bagian utama:

1. **RequestConvolution**: Menangani antarmuka pengguna (UI) dinamis agar user bisa mengatur kernel/matriks.
2. **ProcessConvolutionFast**: Mesin pemrosesan gambar berkinerja tinggi menggunakan manipulasi memori langsung (*Direct Memory Access*) dan *Multithreading*.

Bagian 1: Namespace & Deklarasi Class

C#

```
using System;
using System.Drawing;
using System.Drawing.Imaging;
using System.Runtime.InteropServices;
using System.Threading.Tasks;
using System.Windows.Forms;
using MiniPhotoshop.Logic.Helpers;

namespace MiniPhotoshop.Logic.ImageProcessing
{
    public class Convolution
    {

```

- **Imports:**

- System.Drawing & System.Drawing.Imaging: Untuk manipulasi objek Bitmap dan data piksel.
 - System.Runtime.InteropServices: Untuk Marshal.Copy, yang memungkinkan kita menyalin data dari memori (RAM) gambar ke array byte agar bisa diproses cepat.
 - System.Threading.Tasks: Untuk Parallel.For, memungkinkan pemrosesan gambar menggunakan semua inti CPU (multicore).
 - MiniPhotoshop.Logic.Helpers: Kemungkinan berisi helper class KernelDatabase yang menyimpan preset matriks.
-

Bagian 2: Method RequestConvolution (UI Setup)

Method ini membuat *Dialog Form* secara manual (tanpa Visual Designer) untuk meminta input dari user.

C#

```
public Bitmap RequestConvolution(Bitmap sourceImage)
{
    using (Form inputForm = new Form())
    {
        // ... Konfigurasi Form Dasar (Judul, Ukuran, Posisi) ...
    }
}
```

- **using (Form ...):** Membuat objek Form yang akan otomatis dibuang dari memori setelah selesai (Dispose).
- Form diatur agar muncul di tengah (CenterParent) dan tidak bisa di-resize sembarangan.

1. UI: Ukuran Kernel & Preset

C#

```
// ... Membuat Label dan ComboBox untuk Ukuran Kernel ...
cmbSize.Items.AddRange(new object[] { "3 x 3", "5 x 5", "7 x 7", "Free" });

// ... Membuat ComboBox untuk Tipe Preset (Blur, Sharpen, Edge) ...
cmbType.Items.AddRange(new object[] { "Custom", "Blur (Box)", "Edge (Sobel)", ... });
```

- Bagian ini membuat *dropdown* agar user bisa memilih ukuran matriks (3x3, 5x5, dst) dan jenis efek. Jika user memilih "Blur", nanti angka-angka di matriks akan otomatis terisi.

2. UI: Padding Options

C#

```
cmbPad.Items.AddRange(new object[] { "0 (Hitam)", "255 (Putih)", "Replicate", "Random Color" });
```

- **Penting:** Konvolusi bermasalah di pinggir gambar (karena tidak ada piksel tetangga di sebelah kiri/atas piksel pojok).
- User memilih bagaimana menangani pinggiran tersebut: apakah dianggap hitam, putih, atau menduplikasi piksel terluar (*Replicate*).

3. UI: Grid Matrix Editor

C#

```
DataGridView grid = new DataGridView() { ... };
```

- Membuat tabel input (DataGridView) di mana user bisa mengetik angka-angka matriks konvolusi secara manual (misalnya: -1, 0, 1 untuk deteksi tepi).

4. UI: Divisor & Offset

C#

```
NumericUpDown numDivisor = ...; // Pembagi hasil penjumlahan
NumericUpDown numOffset = ...; // Penambah kecerahan (bias)
```

- **Divisor:** Digunakan untuk menormalisasi hasil. Contoh: Blur 3x3 menjumlahkan 9 piksel, jadi harus dibagi 9 agar gambar tidak menjadi terlalu putih (overexposed).
- **Offset:** Digunakan untuk menggeser nilai warna. Penting untuk filter deteksi tepi yang mungkin menghasilkan nilai negatif (agar bisa terlihat abu-abu/jelas).

5. Logic Event Handlers (Interaksi User)

C#

```
// Event: Setup Grid (Membuat kolom/baris sesuai ukuran kernel)
Action<int> SetupGrid = (n) => { ... };
```

```
// Event: Preset Kernel (Mengisi angka dari KernelDatabase ke Grid)
```

```
cmbType.SelectedIndexChanged += (s, e) => {
    // Ambil data dari KernelDatabase dan masukkan ke Grid, Divisor, dan Offset
    KernelData data = KernelDatabase.GetKernel(cmbType.Text, n);
    // ... loop mengisi grid ...
};
```

- Kode ini membuat form menjadi interaktif. Jika user memilih "Blur", grid akan terisi angka 1 semua, dan divisor jadi 9.

6. Eksekusi (User Klik OK)

C#

```
if (inputForm.ShowDialog() == DialogResult.OK)
{
    // 1. Parsing angka dari Grid ke array 2D double[,] kernel
    // 2. Ambil nilai Divisor, Offset, dan PaddingType
    return ProcessConvolutionFast(sourceImage, kernel, padding, divisor, offset);
}
return null;
```

- Jika user menekan tombol "Proses", data dikumpulkan dan dikirim ke fungsi utama ProcessConvolutionFast.

Bagian 3: Method ProcessConvolutionFast (Core Engine)

Ini adalah "otak" dari kode ini. Menggunakan pointer dan parallel processing agar filter berjalan real-time.

C#

```
public Bitmap ProcessConvolutionFast(Bitmap source, double[,] kernel, PaddingType paddingType,
double divisor, int offset)
{
    if (divisor == 0) divisor = 1; // Mencegah error pembagian dengan nol

    Bitmap srcBmp = new Bitmap(source);
```

```
int width = srcBmp.Width;  
int height = srcBmp.Height;
```

1. LockBits (Akses Memori Langsung)

C#

```
BitmapData srcData = srcBmp.LockBits(  
    new Rectangle(0, 0, width, height),  
    ImageLockMode.ReadOnly,  
    PixelFormat.Format32bppArgb  
);
```

- **LockBits**: Mengunci gambar di memori RAM agar alamat memorinya tidak pindah-pindah. Ini jauh lebih cepat daripada GetPixel().
- **Format32bppArgb**: Format standar 4 byte per piksel (Blue, Green, Red, Alpha).

2. Marshaling (Copy ke Array Byte)

C#

```
int bytes = Math.Abs(srcData.Stride) * height;  
byte[] rgbValues = new byte[bytes];  
byte[] resultValues = new byte[bytes]; // Array kosong untuk hasil
```

```
Marshal.Copy(srcData.Scan0, rgbValues, 0, bytes);
```

- Data piksel disalin dari pointer Scan0 ke array C# biasa (rgbValues) agar lebih aman dan cepat diakses dalam loop.

3. Persiapan Variabel Kernel

C#

```
int kH = kernel.GetLength(0); // Tinggi Kernel
```

```
int kW = kernel.GetLength(1); // Lebar Kernel
int radY = kH / 2; // Radius vertikal (misal 3x3 -> radius 1)
int radX = kW / 2; // Radius horizontal
int stride = srcData.Stride; // Lebar baris data dalam byte
```

- Stride adalah lebar satu baris gambar dalam memori (kadang ada padding byte tambahan di ujung baris untuk alasan performa CPU).

4. Parallel Loop (Multithreading)

C#

```
Parallel.For(0, height,
    () => new Random(Guid.NewGuid().GetHashCode()), // Inisialisasi Random per thread
    (y, loopState, rng) =>
    {
        for (int x = 0; x < width; x++)
        {
            // ... Proses per piksel dimulai di sini ...
        }
    })
```

- **Parallel.For**: Membagi tugas pemrosesan baris (y) ke semua inti CPU yang tersedia. Ini membuat filter berjalan 4x-8x lebih cepat dibanding for biasa.
- **rng**: Objek Random dibuat lokal per thread untuk mencegah *race condition* jika user memilih padding tipe "Random".

5. Algoritma Konvolusi

Rumus matematika konvolusi diterapkan di sini:

$$Baru(x,y) = \sum_{j=-rad}^{rad} \sum_{i=-rad}^{rad} (Piksel(x-i, y-j) \times Kernel(i, j))$$

C#

```
double sumR = 0, sumG = 0, sumB = 0;

for (int ky = -radY; ky <= radY; ky++) // Loop Kernel Baris
{
    // Hitung posisi Y tetangga
    for (int kx = -radX; kx <= radX; kx++) // Loop Kernel Kolom
    {
        // ... Hitung konvolusi ...
    }
}
```

```

int currentY = y + ky;

// ... Logika Padding Y (Cek jika keluar batas atas/bawah) ...
if (currentY < 0) { ... } // Handle batas

for (int kx = -radX; kx <= radX; kx++) // Loop Kernel Kolom
{
    // Hitung posisi X tetangga
    int currentX = x + kx;

    // ... Logika Padding X (Cek jika keluar batas kiri/kanan) ...

```

6. Menangani Padding (Batas Gambar)

Kode ini menangani kasus jika kernel sedang membaca piksel di luar koordinat gambar:

C#

```

if (!isOutY && !isOutX)
{
    // Ambil warna asli dari array byte
    int index = (currentY * stride) + (currentX * 4);
    b = rgbValues[index];
    g = rgbValues[index + 1];
    r = rgbValues[index + 2];
}
else
{
    // Jika di luar batas, gunakan strategi padding
    if (paddingType == PaddingType.White) { r=255; ... }
    else if (paddingType == PaddingType.Random) { ... }
    // Default Replicate ditangani dengan memanipulasi currentX/Y di atas
}

```

7. Kalkulasi Akhir Per Piksel

C#

```

// Akumulasi Bobot
double w = kernel[ky + radY, kx + radX];
sumR += r * w;
sumG += g * w;
sumB += b * w;

// ... Selesai loop kernel ...

// Normalisasi
int finalR = (int)((sumR / divisor) + offset);
// ... Lakukan untuk G dan B ...

// Clamping (Batasi nilai 0 - 255)
// Simpan ke array hasil
int finalIdx = (y * stride) + (x * 4);
resultValues[finalIdx] = (byte)Math.Min(255, Math.Max(0, finalB));
// ...
resultValues[finalIdx + 3] = 255; // Alpha channel (selalu 255/tidak transparan)

```

- **Clamping:** Penting karena hasil matematika bisa negatif atau > 255 (misal pada filter Sharpen). Kita harus memaksanya kembali ke rentang byte (0-255).

8. Finalisasi (Unlock Bits)

C#

```

srcBmp.UnlockBits(srcData); // Lepas kunci memori sumber

Bitmap resultBmp = new Bitmap(width, height, PixelFormat.Format32bppArgb);
BitmapData resData = resultBmp.LockBits(... WriteOnly ...);

// Salin array hasil (resultValues) kembali ke Bitmap baru
Marshal.Copy(resultValues, 0, resData.Scan0, bytes);
resultBmp.UnlockBits(resData);

return resultBmp;

```

- Hasil pemrosesan yang ada di array byte resultValues disalin ke sebuah Bitmap baru (resultBmp) untuk ditampilkan ke layar.

Kesimpulan

Kode ini adalah implementasi standar industri untuk pemrosesan citra di C#:

1. **Fleksibel:** User bisa membuat kernel apa saja (Blur, Sharpen, Emboss, Sobel).
2. **Cepat:** Menggunakan LockBits (hindari GetPixel) dan Parallel.For (multicore).
3. **Aman:** Menangani batas gambar (padding) dan validasi nilai warna (clamping).

Langkah selanjutnya yang bisa saya bantu:

Apakah Anda ingin saya buat isi dari class KernelDatabase agar kode "Preset" di atas (seperti Blur/Sobel) bisa berfungsi sepenuhnya?